

require the use of an extra object for this.

```
inline fun consume(f: () -> Unit): Boolean {
    f()
    return true
}
```

Extension functions

All of us, at some stage, want to add a new function to an existing class but this means modifying the original class and doing some more work on the code. This is where extension functions come to our rescue. As the name suggests, they are used to inject functionality to already existing classes without modifying the original class. People who know C# may be aware of this feature. These functions eliminate the need for creating 'Util' classes.

Here is an example:

```
fun String.isEmail(): Boolean {
    return Patterns.EMAIL_ADDRESS.matcher(this).matches()
}
```

The usage is as follows:

```
val email = "go.kotlin@android.com"
if (email.isEmail()) {
    // Do login
}
```

Extension function with receiver

This feature is unique to Kotlin, and is like a combination of an extension function and a higher order function whereby you pass an extension function as a parameter to another extension function.

```
inline fun SharedPreferences.edit(func: SharedPreferences.
Editor.() -> Unit) {
    val editor = edit()
    editor.func()
    editor.apply()
}

//Usage
val pref = PreferenceManager.getDefaultSharedPreferences(con
```

```
text)
pref.edit{
    putString("first_name", "A")
    putString("last_name", "J")
    remove("last_name")
}
```

Smart cast

Smart casting is done by the Kotlin compiler on our behalf. It even handles the negative scenario, like if the 'view' is not a TextView.

```
// Java way
if (view instanceof TextView) {
    ((TextView) view).setText("Avifors");
}

// Kotlin way
if (view is TextView) {
    // Smart casting
    view.text = "Avifors"
}
```

There are however a few, minor issues with Kotlin. These are:

- Slightly increased build time
 - Adds Dex Count (6K) to your app
 - Difficult to use the power of functional programming as most of us come from object-oriented programming
- That said, if you find Kotlin an awesome language, then these issues won't be a deal breaker for you.

With Kotlin, we can enjoy writing the code. It shrinks the code base and even reduces the number of crashes. There is so much more about Kotlin that hasn't been covered here. For instance, Kotlin v1.1 has more new treats for asynchronous programming.

So go on, get your hands dirty with Kotlin! And have lots of fun writing code. There is great tooling support for Kotlin in IntelliJ and Android Studio (from v3). 

By: Akshay Chordia

The author is an Indie Android Developer and is well versed with the latest mobile technologies. He can be reached at akshaychordiya2@gmail.com

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU